intermediate results don’t have to pass through it, so it’s not a bottleneck for computation. An objection could be that the master node is still a bottleneck for availability, since if the master node crashes the computation can’t be completed: using replicas (either live copies of master-slave), however, we could get availability guarantees through (limited) redundancy.

There are catches, of course. The first one is that not all computations are suitable for the MapReduce model (or for parallel execution altogether). In general, if data entries are somehow connected and scattered pieces of data influence each other’s contribution to the final result, then parallelization could be impossible: time series are a good example of data that normally needs to be processed sequentially, because the final result depends on the sequence of adjacent data.

For MapReduce, requirements are even higher: in order to gain an advantage from applying it, we need data that can be grouped by some attributes/fields, and that can be reduced for each of these groups separately.

The operation performed in reducers, moreover, must be *associative*, so that the order in which the intermediate sub-lists are outputted by mappers must **not** matter.